CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G1
System Integration and Architecture

System Design Document (SDD)

Project Title: Research Center

Prepared By: Uy, Emman Jay Cañaveral

Version: 0.2

Date: February 28, 2026
Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	02/21/26	Uy, Emman Jay C.	Initial document	Final
0.2	2/28/26	Uy, Emman Jay C.	Final document	Final

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 4

1.0 INTRODUCTION 5

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5

3.0 NON-FUNCTIONAL REQUIREMENTS 8

4.0 SYSTEM ARCHITECTURE 9

5.0 API CONTRACT & COMMUNICATION 10

Authentication Endpoints 11

User Registration 11

User Login 11

6.0 DATABASE DESIGN 13

7.0 UI/UX DESIGN 14

8.0 PLAN 17

EXECUTIVE SUMMARY

1.1 Project Overview

ResearchCenter is a real-time collaborative platform designed for students, researchers, and academic groups to create shared repositories for storing, organizing, and sharing research materials. The system enables users to create individual or group repositories, invite members, and collaboratively manage PDFs, study links, and references to streamline capstone, thesis, and research workflows. The platform integrates a Spring Boot backend API, a React web application built with Vite, and a native Android mobile application built with Kotlin and XML.

1.2 Objectives

1. Develop a collaborative research platform where users can create repositories and invite members to share academic materials.
2. Implement a three-tier architecture using Spring Boot (backend), React with Vite (web), and Android (mobile) with Supabase for database and storage.
3. Create RESTful APIs for communication between all system components, protected by JWT and Role-Based Access Control with User and Admin roles.
4. Design a responsive user interface using Tailwind CSS for web and XML layouts for mobile that works consistently across platforms.
5. Integrate external services including Google Books API for metadata fetching, Google OAuth for social login, and SendGrid for SMTP email notifications (welcome, invitations, requests, fulfillment).
6. Deploy database and storage to Supabase (free tier), backend to Render (free tier), and frontend to Vercel (free tier) for production readiness.

1.3 Scope

Included Features:

- User registration and login via Email/Password (BCrypt) and Google OAuth
- JWT-based authentication with protected routes and /me endpoint
- Role-Based Access Control (User and Administrator roles)
- Repository creation and management (create, edit, delete repositories)
- Repository membership (invite members, accept invitations, manage members)
- Material management within repositories (upload PDFs, add links/references)

- Material Request & Fulfillment system within repositories
- Research Tags and Status Labels (e.g., "To Read", "In Progress", "Completed")
- Bookmarks for saving favorite materials
- Google Books API integration for optional automatic metadata fetching (for books with ISBN)
- SendGrid for SMTP email notifications (welcome, invitations, new requests, fulfillment)
- File upload and storage for PDFs in Supabase Storage (max 10MB)
- Activity logging for tracking user actions within repositories
- Real-time updates via WebSocket for collaborative features
- Responsive web interface and native Android mobile application with XML layouts
- Admin dashboard for user and repository management

Excluded Features:

- Real payment processing
- Advanced analytics dashboard
- Machine learning recommendations
- Integration with external reference managers (Zotero, Mendeley)
- Integration with external LMS (Canvas, Blackboard, etc.)
- Video conferencing or chat features
- Public repository discovery

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the ResearchCenter system. It provides detailed requirements, architectural decisions, API contracts, database design, and implementation roadmap to guide development and ensure all components integrate seamlessly. The document defines how the Spring Boot backend, React web application, and Android mobile application will work

together to create a collaborative research platform where users can create repositories, invite members, and share academic materials.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: ResearchCenter
Domain: Academic Collaboration / Research Management.
Primary Users:

1. Researchers - Students, master's students, PhD candidates, individual researchers
2. Research Groups - Capstone teams, thesis groups, lab teams
3. Administrators - System administrators who manage users and content

Problem Statement: Researchers face scattered data across multiple platforms (Google Drive, email attachments, reference managers) and difficulty tracking shared resources within groups. There is no centralized system designed specifically for academic research workflows that enables organized storage, collaborative sharing, and invitation-based group repositories.

Solution: A collaborative platform built around private, invitation-only repositories where users can create spaces for different projects, invite team members, and collaboratively manage research materials with real-time updates and automated metadata fetching. All repositories are private by default and accessible only to the owner and invited members. Users can only see repositories they own or have been invited to, with no public discovery of other users' repositories. Administrators have elevated privileges to view all repositories for moderation purposes only, protecting ongoing research privacy.

2.2 Core User Journeys

Journey 1: First-time User Registration & Repository Creation

1. User opens web application or mobile app
2. Clicks "Sign Up" and creates account via email/password or Google OAuth
3. Receives welcome email via SMTP with getting started guide
4. Upon first login, user is prompted to create their first Repository
5. User creates repository with:
    - Repository name (e.g., "Thesis 2026", "ML Research Group", "Capstone Project")

    - Description
6. Repository is created and user automatically becomes Owner of the repository
7. User is redirected to repository dashboard showing materials, requests, and members

Journey 2: Inviting Members to Repository

1. Repository owner navigates to repository and clicks "Members" tab
2. Clicks "Invite Member" and enters email addresses of colleagues
3. System sends invitation emails to each email address
4. Recipients receive email with:
    - Repository name and inviter name
    - Link to join (if they have an account) or link to register (if new)
5. When recipient clicks link and accepts, they become a Member of the repository
6. Owner receives notification that new member has joined
7. New members can now view/add materials and requests within the repository

Journey 3: Adding Materials to Repository

1. User (owner or member) navigates to repository
2. Clicks "Add Material"
3. User can either:
    - - Enter ISBN or title and click "Fetch Metadata" to auto-populate fields (for books)
    - - Or manually enter all material details (for articles, papers, etc.)
4. User selects material type:
    - - PDF - upload file from device (stored in Supabase Storage)
    - - Link - enter URL to external resource
    - - Reference - manually enter details
5. User assigns tags (e.g., "machine-learning", "thesis") and status label (To Read/In Progress/Completed)
6. Material is saved to repository
7. Email notification can be sent to repository members about new material
8. Activity log records the upload action

Journey 4: Requesting Materials from Group

1. User navigates to repository's "Requests" board
2. Clicks "New Request" and fills in:
    - Title (e.g., "Looking for: Deep Learning Book by Goodfellow")
    - Description (details about what's needed and why)
3. System creates request with status "OPEN"
4. Email notification sent to all repository members about new request
5. Request appears in real-time on all members' screens via WebSocket

6. Members can view all open requests and comment if needed

Journey 5: Fulfilling a Request

1. Repository member sees open request and clicks "Fulfill"
2. Member provides the requested material by:
    - Uploading a PDF file
    - Adding a link to the resource
    - Creating a reference entry
3. System links the provided material to the request
4. Request status updates to "FULFILLED" in real-time (WebSocket)
5. Email notification sent to requester:
6. "Your request for [title] has been fulfilled by [member name]"
7. Link to view the material
8. Activity log records the fulfillment
9. All members can see the fulfilled request and access the material

Journey 6: Managing Multiple Repositories

1. User logs in and lands on their Repositories Dashboard
2. User sees list of all repositories they own or are members of
3. For each repository, user can see:
    - Repository name and description
    - Number of materials
    - Number of members
    - Recent activity
4. User can click into any repository to view its contents
5. User can create new repositories for different projects
6. User can leave repositories they no longer need to be part of

Journey 7: Administrator Managing the System

1. Admin logs in with admin credentials
2. Navigates to Admin Dashboard (separate route from regular user views)
3. User Management:
    - View all users in system with pagination
    - Change user roles (promote to Admin, demote to User)
    - Suspend/activate user accounts
4. Repository Management:
    - View all repositories across the system (for moderation purposes)
    - View repository details including owner and member count
    - Delete inappropriate repositories
    - View repository statistics

5. Content Moderation:
    - View all materials across all repositories
    - Remove inappropriate content
6. System Statistics:
    - Total users, repositories, materials, requests
    - Storage usage

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (register, login, logout, JWT, /me endpoint)
2. Google OAuth login
3. Role-Based Access Control (User and Admin roles)
4. Repository management (create, view, edit, delete)
5. Repository membership (invite members, accept invitations)
6. Material management (add, view, edit, delete PDFs/links/references)
7. File upload (PDF to Supabase Storage)
8. Material request system (create request, fulfill request, track status)
9. Google Books API integration (fetch metadata by ISBN/title)
10. SendGrid for SMTP email notifications (welcome, invitations, new requests, fulfillment)
11. Real-time updates via WebSocket for requests
12. Admin panel (view all users, change roles, view all repositories for moderation, delete inappropriate content)

SHOULD HAVE

1. Research tags for materials
2. Status labels (To Read, In Progress, Completed)
3. Bookmarks for saving materials
4. Activity log for tracking actions
5. Responsive web design with Tailwind CSS
6. XML-based Android layouts (no Jetpack Compose)

COULD HAVE

1. Quick notes attached to materials
2. Search filtering by tags and status
3. Export references to BibTeX format

WON'T HAVE

1. Real payment processing

2. Advanced analytics dashboard
3. Video conferencing or chat
4. Integration with reference managers (Zotero, Mendeley, etc.)
5. Integration with external LMS (Canvas, Blackboard, etc.)
6. Public repository discovery

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login (Web and Mobile)
- **Fields:** Email, Password, First Name, Last Name
- **Validation:** Email format, password strength (min 8 chars), required fields
- **Security:** JWT tokens (1-hour expiration), refresh tokens (7 days), BCrypt hashing, HTTPS only
- **Integrations:** Google OAuth, SMTP email for welcome
- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout, POST /auth/refresh, GET /users/me

Feature: Role-Based Access Control

- **Roles:** User, Admin
- **User API Access:** /api/repositories/* (owned/member), /api/materials/* (in repos), /api/requests/*, /api/users/me
- **User UI Access:** Repositories Dashboard, Repository views, Materials, Requests, Profile
- **Admin API Access:** All endpoints including /api/admin/*
- **Admin UI Access:** All User views + Admin Dashboard, User Management, Repository Management

Feature: Repository Management

- **Screens:** Create Repository, Repository Settings
- **Functions:** Create repository, edit details, delete repository (owner only)
- **Fields:** Name (required), Description (optional)
- **API Endpoints:** POST /repositories, GET /repositories, PUT /repositories/{id}, DELETE /repositories/{id}
- **Access Control:** Owner has full access; Members have view access; Others have no access aside Admin

Feature: Repository Membership Management

- **Screens:** Members tab, Invite Member form
- **Functions:** Invite member via email, accept invitation, remove member, change role
- **Email Triggers:** Invitation email, new member joined notification

- **API Endpoints:** POST /repositories/{id}/invite, POST /invitations/accept, DELETE /repositories/{id}/members/{userId}, PUT /repositories/{id}/members/{userId}/role
- **Constraints:** One user per repository (no duplicates)

Feature: Material Management

- **Screens:** Materials tab, Add Material form, Material Detail view
- **Functions**: Add material, view materials, edit metadata, delete material
- **Material Types:** PDF (upload), Link (URL), Reference (manual)
- **File Upload:** PDF to Supabase Storage (max 10MB)
- **External API:** Google Books API for metadata fetching
- **API Endpoints:** POST /materials, GET /materials, GET /materials/{id}, PUT /materials/{id}, DELETE /materials/{id}

Feature: Material Request & Fulfillment

- **Screens:** Requests board, New Request form, Fulfill Request form
- **Functions:** Create request, fulfill request, track status
- **Statuses:** OPEN, PENDING, FULFILLED, CANCELLED
- **Real-time:** WebSocket updates on request creation and fulfillment
- **Email Triggers:** New request email to members, fulfillment email to requester
- **API Endpoints:** POST /requests, GET /requests, POST /requests/{id}/fulfill, GET /requests/{id}

Feature: Google Books API Integration

- **Endpoint:** https://www.googleapis.com/books/v1/volumes?q=isbn:{isbn} or ?q=intitle:{title}
- **Trigger:** User enters ISBN or title, clicks "Fetch Metadata"
- **Data Retrieved:** title, authors, publisher, publishedDate, description, pageCount, categories, thumbnail
- **Usage:** Auto-populate material creation form, save metadata to database

Feature: Email Notifications (SendGrid SMTP)

- **Welcome Email:** After successful registration (new user)
- **Invitation Email:** Owner invites member (invited email)
- **New Member Joined:** Member accepts invitation (repository owner)
- **New Request:** Request created (all repository members)
- **Request Fulfilled:** Request fulfilled (requester)

Feature: Admin Dashboard

- **Screens:** Admin Dashboard, User Management, Repository Moderation
- **Functions**: View all users, change user roles, view all repositories, delete inappropriate content

- **Access Control:** Admin role required
- **API Endpoints:** GET /admin/users, PUT /admin/users/{id}/role, GET /admin/repositories, DELETE /admin/materials/{id}

2.5 Acceptance Criteria

AC-1: Successful User Registration

- ➢ Given I am a new user
- ➢ When I enter valid email and strong password
- ➢ And enter first name and last name
- ➢ And click "Create Account"
- ➢ Then my password should be hashed with BCrypt and stored in database
- ➢ And I should receive a JWT token
- ➢ And I should receive a welcome email via SMTP
- ➢ And I should be redirected to create my first repository\

AC-2: Google OAuth Login

- ➢ Given I am on the login page
- ➢ When I click "Login with Google"
- ➢ And select my Google account
- ➢ Then I should be authenticated
- ➢ And a new user record should be created if first time
- ➢ And I should be redirected to my repositories dashboard

AC-3: /me Endpoint Returns Current User

- ➢ Given I am logged in with valid JWT token
- ➢ When I make a GET request to /users/me
- ➢ Then I should receive my user information (id, email, name, role)
- ➢ And my password should NOT be included in the response

AC-4: Create Repository

- ➢ Given I am logged in
- ➢ When I click "Create New Repository"
- ➢ And enter repository name and description
- ➢ And click "Create"
- ➢ Then a new repository should be created
- ➢ And I should be set as the Owner
- ➢ And I should be redirected to the repository dashboard

AC-5: Invite Member to Repository

- ➢ Given I am the owner of a repository

- When I click "Invite Member" and enter an email address
- Then an invitation email should be sent to that address
- And the invitation should be recorded in the system

AC-6: Accept Repository Invitation

- Given I receive an invitation email
- When I click the join link in the email
- And I am logged in (or register first)
- Then I should become a Member of the repository
- And the repository owner should receive a notification

AC-7: Add Material with Google Books API

- Given I am in a repository
- When I click "Add Material"
- Then I should have two options:
    - - Option A: Enter ISBN/title and click "Fetch Metadata" to auto-populate fields
    - - Option B: Manually enter all material details
- And I can upload a PDF, add a link, or create a reference
- And the material should be saved to the repository

AC-8: Add Material with Google Books API

- Given I am a member of a repository
- When I navigate to the Requests board
- And click "New Request" and enter title and description
- Then a new request should be created with status "OPEN"
- And email notifications should be sent to all repository members
- And the request should appear in real-time on their screens

AC-9: Fulfill Material Request

- Given there is an OPEN request in a repository I belong to
- When I click "Fulfill" and provide the requested material
- Then the request status should update to "FULFILLED"
- And the requester should receive an email notification
- And the material should be linked to the request
- And all members should see the update in real-time

AC-10: View Repositories Dashboard

- Given I am logged in
- When I navigate to the home page
- Then I should see a list of all repositories I own or am a member of
- And for each repository, I should see name, description, and member count

- ➢ And I can click into any repository to view its contents

AC-11: Role-Based Access - Admin Dashboard

- ➢ Given I am logged in as an Admin
- ➢ When I navigate to /admin/dashboard
- ➢ Then I should see user management interface
- ➢ And I can view all users, change roles, and suspend accounts
- ➢ And I can view all repositories across the system

AC-12: Role-Based Access - User Restriction

- ➢ Given I am logged in as a regular User
- ➢ When I attempt to access /admin/dashboard
- ➢ Then I should receive a 403 Forbidden response
- ➢ And the UI should hide admin functions

AC-13: Protected Page Access Without Token

- ➢ Given I am not logged in
- ➢ When I attempt to access any repository page
- ➢ Then I should be redirected to the login page

AC-14: User Logout

- ➢ Given I am logged in
- ➢ When I click "Logout"
- ➢ Then my token should be cleared from storage
- ➢ And I should be redirected to the login page
- ➢ And I can no longer access protected pages

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests
- Web page load time: ≤ 3 seconds on broadband
- Mobile app cold start: ≤ 3 seconds
- Support 100 concurrent users
- Database queries complete within 500ms
- File upload: Handle up to 10MB files within 5 seconds
- Real-time updates: WebSocket messages delivered within 2 seconds

3.2 Security Requirements

- HTTPS for all communications (TLS 1.2+)

- JWT token authentication with 1-hour expiration
- Refresh tokens with 7-day expiration stored in database
- Password hashing with BCrypt (salt rounds = 12)
- SQL injection prevention via parameterized queries
- XSS protection
- Rate limiting: 100 requests/minute per IP
- Admin endpoints require role verification
- File validation: PDF only, max 10MB
- Secure token storage: HTTP-only cookies for web, EncryptedSharedPreferences for Android

3.3 Compatibility Requirements

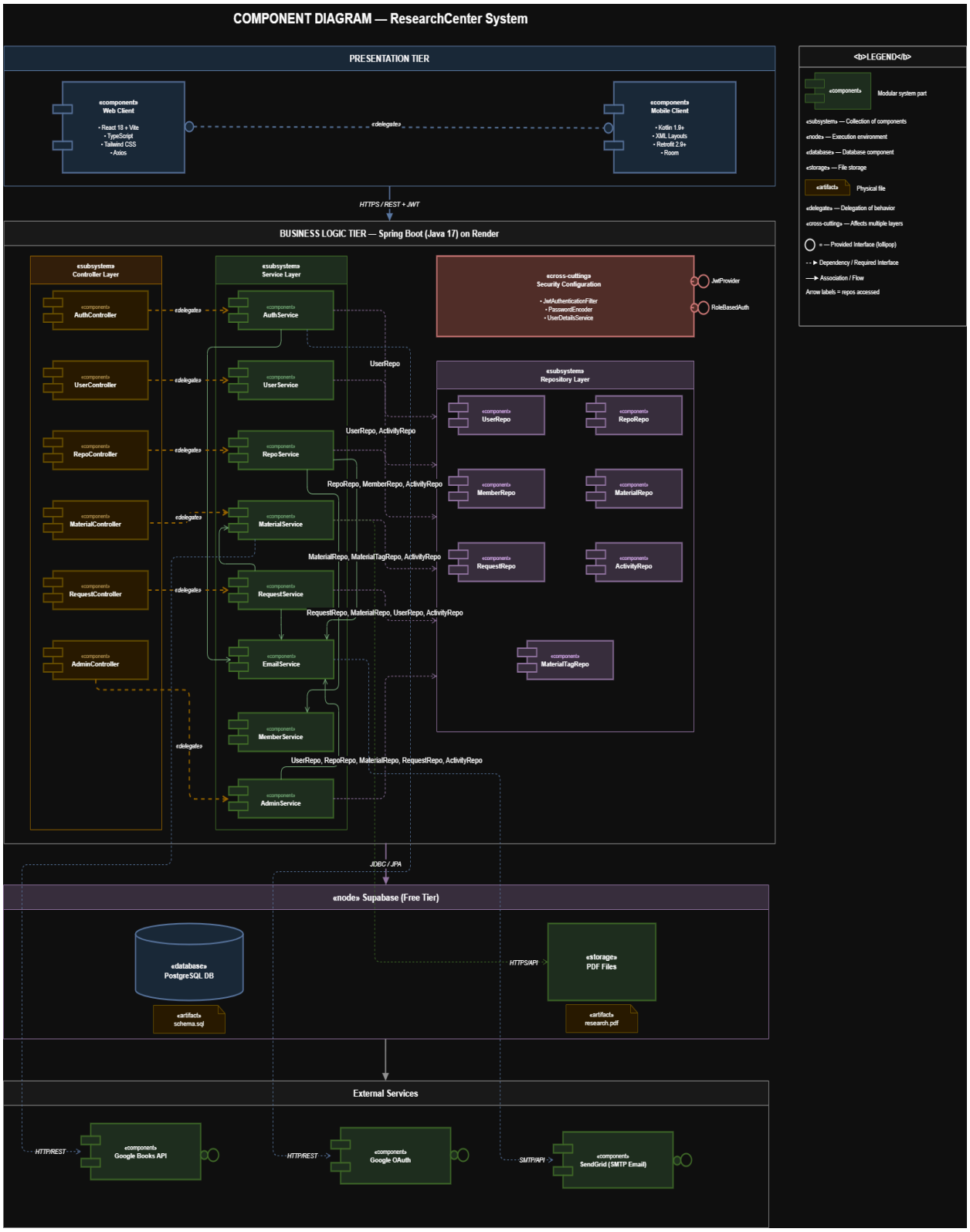
- **Web Browsers:** Chrome 90+, Firefox 88+, Safari 14+, Edge 90+ (latest 2 versions)
- **Android:** API Level 24+ (Android 7.0+)
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+

3.4 Usability Requirements

- Complete first repository creation within 5 minutes for new users
- WCAG 2.1 Level AA compliance for web
- Consistent navigation across all pages
- Clear error messages with recovery options
- Touch targets minimum 44x44px on mobile
- Keyboard navigation support
- Loading states for async operations
- Empty states with guidance for first-time users

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA, Spring WebSocket, JJWT
- **Database:** PostgreSQL 14+ (Supabase)
- **Storage:** Supabase Storage (1 GB free tier)
- **Email Service:** SendGrid (SMTP)
- **Web Frontend:** React 18, Vite, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, XML Layouts, Retrofit, Room
- **Build Tools:** Maven (Backend), npm (Web), Gradle (Android)
- **Deployment:** Render (Backend), Vercel (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[backend-url].onrender.com/api/v1 (production) http://localhost:8080/api/v1 (development)
Format	JSON for all requests and responses
Authentication	Bearer token (JWT) in Authorization header: Authorization: Bearer <token>

Response Structure	{
	"success": boolean,
	"data": object|null,
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST

Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"email": <email>,
	"password": <password>,
	"firstname": <firstname>,
	"lastname": <lastname>
	}

Response Structure	{
	"success": boolean,
	"data": {
	"user": {
	"id": <userId>,
	"email": <email>,
	"firstname": <firstname>,
	"lastname": <lastname>,
	"role": <role>
	},
	"accessToken": <token>,
	"refreshToken": <token>
	} | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"email": <email>,
	"password": <password>
	}

Response Structure	{
	"success": boolean,
	"data": {
	"user": {
	"id": <userId>,
	"email": <email>,

	"firstname": <firstname>,
	"lastname": <lastname>,
	"role": <role>
	},
	"accessToken": <token>,
	"refreshToken": <token>
	} | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

User Logout

Description	User Logout
API URL	/auth/logout
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token

Request Payload	{
	"refreshToken": <token>
	}

Response Structure	{
	"success": boolean,
	"data": {
	"message": "Successfully logged out"
	} | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

User Endpoints

Get Current User (/me)

Description	Get authenticated user information
API URL	/users/me
HTTP Request Method	GET
Format	JSON for all requests/responses

Authentication	Bearer Token
Request Payload	None

Response Structure	{
	"success": boolean,
	"data": {
	"id": <userId>,
	"email": <email>,
	"firstname": <firstname>,
	"lastname": <lastname>,
	"role": <role>,
	"createdAt": <timestamp>
	} | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

Repository Endpoints

Create Repository

Description	Create a new repository
API URL	/repositories
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token

Request Payload	{
	"name": <repositoryName>,
	"description": <description>
	}

Response Structure	{
	"success": boolean,
	"data": {
	"id": <repositoryId>,
	"name": <repositoryName>,
	"description": <description>,
	"ownerId": <userId>,
	"createdAt": <timestamp>,
	"updatedAt": <timestamp>
	} | null,
	"error": {
	"code": string,
	"message": string,

	"details": object | null } | null, "timestamp": string }

Get User Repositories

Description	Get all repositories user owns or is member of
API URL	/repositories
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token
Request Payload	None

Response Structure	{
	"success": boolean,
	"data": [
	{
	"id": <repositoryId>,
	"name": <repositoryName>,
	"description": <description>,
	"ownerId": <userId>,
	"role": <roleInRepo>,
	"memberCount": <count>,
	"materialCount": <count>,
	"createdAt": <timestamp>
	}
	] | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

Get Repository by ID

Description	Get specific repository details
API URL	/repositories/{id}
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token
Request Payload	None

Response Structure	{
	"success": boolean,
	"data": {

	"id": <repositoryId>,
	"name": <repositoryName>,
	"description": <description>,
	"ownerId": <userId>,
	"ownerName": <fullName>,
	"members": [
	{
	"userId": <userId>,
	"name": <fullName>,
	"role": <roleInRepo>,
	"joinedAt": <timestamp>
	}
	],
	"memberCount": <count>,
	"materialCount": <count>,
	"createdAt": <timestamp>,
	"updatedAt": <timestamp>
	} | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

Update Repository

Description	Update repository details
API URL	/repositories/{id}
HTTP Request Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer Token (Owner only)

Request Payload	{
	"name": <repositoryName>,
	"description": <description>
	}

Response Structure	{
	"success": boolean,
	"data": {
	"id": <repositoryId>,
	"name": <repositoryName>,
	"description": <description>,
	"ownerId": <userId>,
	"updatedAt": <timestamp>
	} | null,

	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

Delete Repository

Description	Delete a repository
API URL	/repositories/{id}
HTTP Request Method	DELETE
Format	JSON for all requests/responses
Authentication	Bearer Token (Owner only)
Request Payload	None

Response Structure	{
	"success": boolean,
	"data": {
	"message": "Repository deleted successfully"
	} | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

Member Endpoints

Invite Member to Repository

Description	Invite a user to join repository
API URL	/repositories/{id}/invite
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token

Request Payload	{
	"email": <email>
	}

Response Structure	{
	"success": boolean,
	"data": {
	"message": "Invitation sent successfully"

	} | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

List Repository Members

Description	Get all members of a repository
API URL	/repositories/{id}/members
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token
Request Payload	None

Response Structure	{
	"success": boolean,
	"data": [
	{
	"userId": <userId>,
	"name": <fullName>,
	"email": <email>,
	"role": <roleInRepo>,
	"joinedAt": <timestamp>
	}
	] | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

Remove Member from Repository

Description	Remove a member from repository
API URL	/repositories/{id}/members/{userId}
HTTP Request Method	DELETE
Format	JSON for all requests/responses
Authentication	Bearer Token (Owner only)
Request Payload	None

Response Structure	{
	"success": boolean,
	"data": {

	"message": "Member removed successfully"
	} | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

Material Endpoints

Add Material to Repository

Description	Add material to repository
API URL	/materials
HTTP Request Method	POST
Format	JSON for all requests/responses (or multipart/form-data for PDF)
Authentication	Bearer Token

Request Payload	**Request Payload (Link/Reference):**
	{
	"repositoryId": <repositoryId>,
	"title": <title>,
	"description": <description>,
	"materialType": <"LINK" | "REFERENCE">,
	"url": <url>, (for LINK)
	"authors": <authors>, (for REFERENCE)
	"publisher": <publisher>, (for REFERENCE)
	"year": <year>, (for REFERENCE)
	"tags": <tags>,
	"status": <status>
	}
	Request Payload (PDF):
	Form Data:
	- repositoryId: <repositoryId>
	- title: <title>
	- description: <description>
	- materialType: "PDF"
	- file: <file>
	- tags: <tags>
	- status: <status>mestamp": string
	}

Response Structure	{
	"success": boolean,
	"data": {
	"id": <materialId>,
	"repositoryId": <repositoryId>,
	"title": <title>,
	"description": <description>,
	"materialType": <type>,
	"fileUrl": <url>,
	"uploaderId": <userId>,
	"uploaderName": <fullName>,
	"tags": [<tags>],
	"status": <status>,
	"createdAt": <timestamp>
	} | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

List Materials in Repository

Description	Get all materials in repository
API URL	/repositories/{id}/materials
HTTP Request Method	GET
Format	JSON for all requests/responses (or multipart/form-data for PDF)
Authentication	Bearer Token
Request Payload	None
Response Structure	{ "success": boolean, "data": [{ "id": <materialId>, "title": <title>, "description": <description>, "materialType": <type>, "fileUrl": <url>, "uploaderId": <userId>, "uploaderName": <fullName>, "tags": [<tags>], "status": <status>, "createdAt": <timestamp>

	}
	] | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

Get Material by ID

Description	Get specific material details
API URL	/materials/{id}
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token
Request Payload	None

Response Structure	{
	"success": boolean,
	"data": {
	"id": <materialId>,
	"repositoryId": <repositoryId>,
	"repositoryName": <repositoryName>,
	"title": <title>,
	"description": <description>,
	"materialType": <type>,
	"fileUrl": <url>,
	"uploaderId": <userId>,
	"uploaderName": <fullName>,
	"tags": [<tags>],
	"status": <status>,
	"metadata": <googleBooksData> | null,
	"createdAt": <timestamp>,
	"updatedAt": <timestamp>
	} | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

Delete Material

Description	Delete a material
API URL	/materials/{id}
HTTP Request Method	DELETE
Format	JSON for all requests/responses
Authentication	Bearer Token (Owner or uploader)
Request Payload	None

Response Structure	{
	"success": boolean,
	"data": {
	"message": "Material deleted successfully"
	} | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

Request Endpoints

Create Request

Description	Create material request in repository
API URL	/requests
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token

Request Payload	{
	"repositoryId": <repositoryId>,
	"title": <title>,
	"description": <description>
	}

Response Structure	{
	"success": boolean,
	"data": {
	"id": <requestId>,
	"repositoryId": <repositoryId>,
	"title": <title>,
	"description": <description>,
	"requesterId": <userId>,
	"requesterName": <fullName>,
	"status": "OPEN",

	"createdAt": <timestamp>
	} | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

List Requests in Repository

Description	Get all requests in repository
API URL	/repositories/{id}/requests
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token
Request Payload	None

Response Structure	{
	"success": boolean,
	"data": [
	{
	"id": <requestId>,
	"title": <title>,
	"description": <description>,
	"requesterId": <userId>,
	"requesterName": <fullName>,
	"status": <status>,
	"fulfilledBy": <userId> | null,
	"fulfilledByName": <fullName> | null,
	"fulfilledAt": <timestamp> | null,
	"materialId": <materialId> | null,
	"createdAt": <timestamp>
	}
	] | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

Get Request by ID

Description	Get specific request details

API URL	/requests/{id}
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token
Request Payload	None

Response Structure	{
	"success": boolean,
	"data": {
	"id": <requestId>,
	"repositoryId": <repositoryId>,
	"repositoryName": <repositoryName>,
	"title": <title>,
	"description": <description>,
	"requesterId": <userId>,
	"requesterName": <fullName>,
	"status": <status>,
	"fulfilledBy": <userId> | null,
	"fulfilledByName": <fullName> | null,
	"fulfilledAt": <timestamp> | null,
	"materialId": <materialId> | null,
	"materialTitle": <title> | null,
	"createdAt": <timestamp>,
	"updatedAt": <timestamp>
	} | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

Fulfill Request

Description	Fulfill an open request
API URL	/requests/{id}/fulfill
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token

Request Payload	{
	"materialId": <materialId>
	}

Response Structure	{
	"success": boolean,
	"data": {

	"id": <requestId>,
	"status": "FULFILLED",
	"fulfilledBy": <userId>,
	"fulfilledByName": <fullName>,
	"fulfilledAt": <timestamp>,
	"materialId": <materialId>
	} | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

Admin Endpoints

List All Users (Admin)

Description	Get all users with pagination
API URL	/admin/users
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token (Admin only)
Request Payload	None

Response Structure	{
	"success": boolean,
	"data": {
	"users": [
	{
	"id": <userId>,
	"email": <email>,
	"firstname": <firstname>,
	"lastname": <lastname>,
	"role": <role>,
	"createdAt": <timestamp>
	}
	],
	"totalCount": <count>,
	"page": <pageNumber>,
	"totalPages": <totalPages>
	} | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null

	} | null, "timestamp": string }

Change User Role (Admin)

Description	Change user role
API URL	/admin/users/{id}/role
HTTP Request Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer Token (Admin only)

Request Payload	{
	"role": "ADMIN" | "USER"
	}

Response Structure	{
	"success": boolean,
	"data": {
	"id": <userId>,
	"email": <email>,
	"role": <newRole>,
	"updatedAt": <timestamp>
	} | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

Delete Material (Admin)

Description	Delete any material (moderation)
API URL	/admin/materials/{id}
HTTP Request Method	DELETE
Format	JSON for all requests/responses
Authentication	Bearer Token (Admin only)
Request Payload	None

Response Structure	{
	"success": boolean,
	"data": {
	"message": "Material deleted successfully"
	} | null,
	"error": {
	"code": string,
	"message": string,

	"details": object | null } | null, "timestamp": string }

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created successfully
- 400 Bad Request - Invalid input or validation error
- 401 Unauthorized - Authentication required or failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example 1: Invalid Credentials	{
	"success": false,
	"data": null,
	"error": {
	"code": "AUTH-001",
	"message": "Invalid credentials",
	"details": "Email or password is incorrect"
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

Example 2: Validation Failed	{
	"success": false,
	"data": null,
	"error": {
	"code": "VALID-001",
	"message": "Validation failed",
	"details": {
	"email": "Email is required",
	"password": "Must be at least 8 characters"
	}
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

Example 3: Repository Not Found	{
	"success": false,
	"data": null,
	"error": {
	"code": "REPO-001",
	"message": "Repository not found",
	"details": "Repository with ID 123 does not exist"
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

Example 4: Not Repository Owner	{
	"success": false,
	"data": null,
	"error": {
	"code": "REPO-002",
	"message": "Not repository owner",
	"details": "Only the owner can perform this action"
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

Example 5: File Too Large
 	
1
	{
	"success": false,
	"data": null,
	"error": {
	"code": "FILE-001",
	"message": "File too large",
	"details": "File exceeds 10MB limit"
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

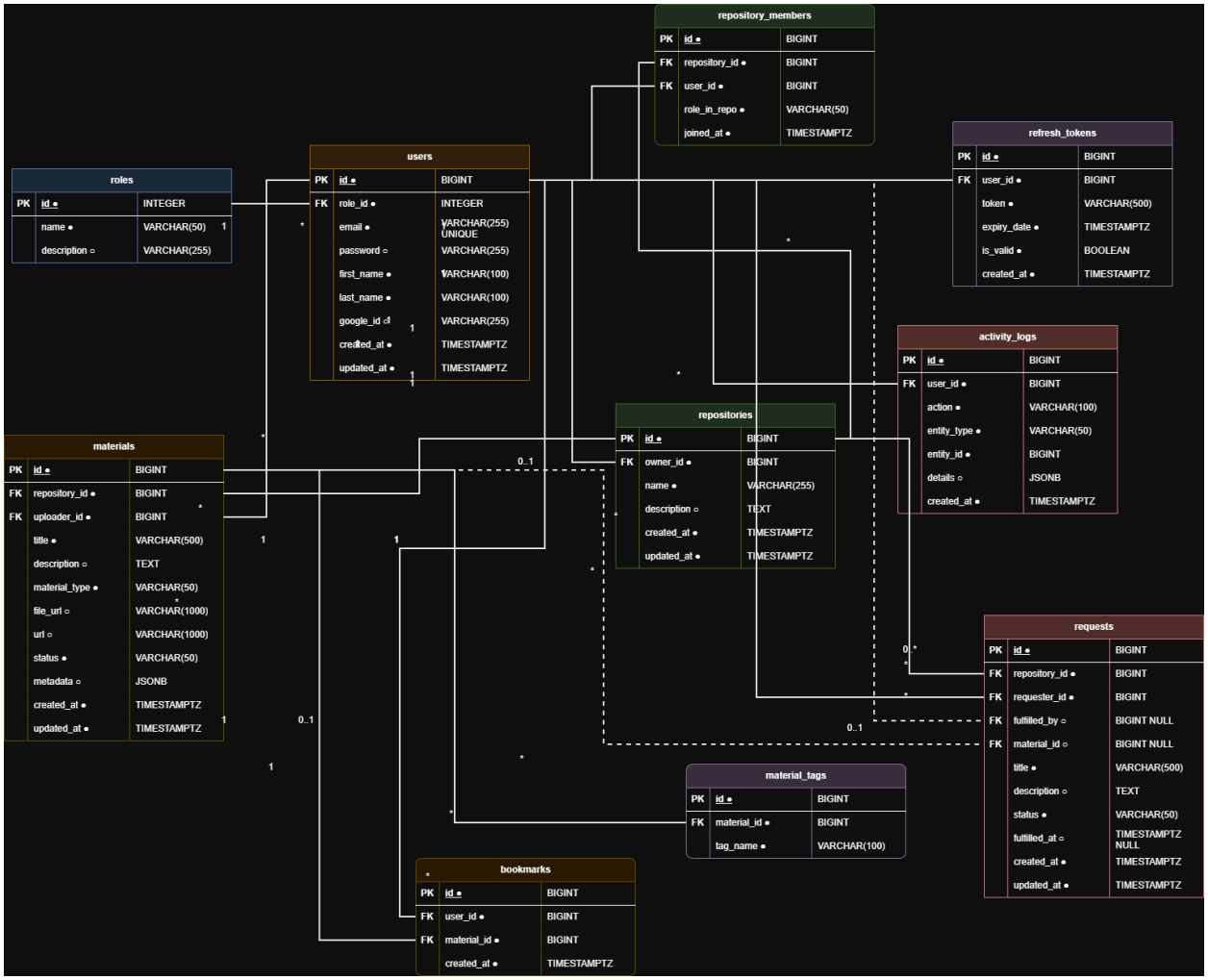
Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Email already exists
- AUTH-003: Token expired
- AUTH-004: Insufficient permissions
- VALID-001: Validation failed
- REPO-001: Repository not found
- REPO-002: Not repository owner
- REPO-003: Not repository member
- MEMBER-001: User already member
- MAT-001: Material not found

- REQ-001: Request not found
- REQ-002: Request already fulfilled
- FILE-001: File too large
- FILE-002: Invalid file type
- DB-001: Resource not found
- DB-002: Duplicate entry
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity



Relationship Diagram

Detailed Relationships:

- **One-to-Many:** User → Repositories (User can own multiple repositories)

- **Many-to-Many:** User ↔ Repository (Users can be members of many repositories via repository_members)
- **One-to-Many: Repository** → Materials (Repository contains multiple materials)
- **One-to-Many: Repository** → Requests (Repository can have multiple requests)
- **One-to-Many: User** → Materials (User can upload multiple materials)
- **One-to-Many: User** → Requests (User can create multiple requests as requester)
- **One-to-Many: User** → Requests (User can fulfill multiple requests as fulfiller)
- **One-to-Many: User** → Bookmarks (User can bookmark many materials)
- **One-to-Many: User** → ActivityLogs (User generates many activity logs)
- **One-to-Many: User** → RefreshTokens (User can have multiple refresh tokens)
- **One-to-Many: Materials** → MaterialTags (Material can have multiple tags)
- **Many-to-One: Materials** → Repository (Materials belong to one repository)
- **Many-to-One: Materials** → User (Materials have one uploader)
- **Many-to-One: Requests** → Repository (Requests belong to one repository)
- **Many-to-One: Requests** → User (Requests have one requester)
- **Many-to-One: Requests** → User (Requests have one fulfiller, if fulfilled)
- **Many-to-One: Requests** → Materials (Requests reference one material, if fulfilled)
- **Many-to-One: Bookmarks** → User (Bookmarks belong to one user)
- **Many-to-One: Bookmarks** → Material (Bookmarks reference one material)

Key Tables:

1. **users -** User accounts and authentication
2. **roles -** User roles (USER, ADMIN)
3. **repositories -** Research repositories created by users
4. **repository_members -** Membership mapping between users and repositories
5. **materials -** Research materials (PDFs, links, references)
6. **material_tags -** Tags associated with materials
7. **requests -** Material requests within repositories
8. **bookmarks -** User bookmarks for materials
9. **activity_logs -** Audit log of user actions
10. **refresh_tokens -** JWT refresh tokens

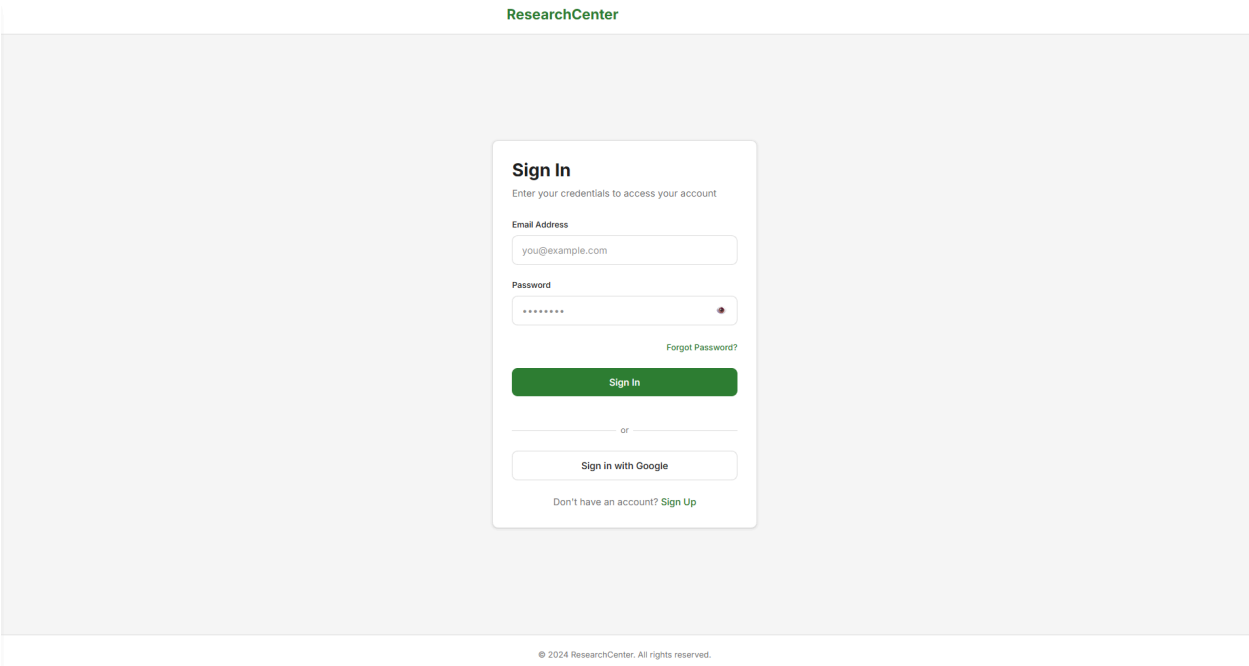
Table Structure Summary:

- **users:** id, email, password, first_name, last_name, role_id, created_at, updated_at, google_id
- **roles:** id, name, description
- **repositories:** id, name, description, owner_id, created_at, updated_at
- **repository_members:** id, repository_id, user_id, role_in_repo, joined_at

- **materials:** id, repository_id, title, description, material_type, file_url, url, uploader_id, status, metadata, created_at, updated_at
- **material_tags:** id, material_id, tag_name
- **requests:** id, repository_id, title, description, requester_id, status, fulfilled_by, fulfilled_at, material_id, created_at, updated_at
- **bookmarks:** id, user_id, material_id, created_at
- **activity_logs:** id, user_id, action, entity_type, entity_id, details, created_at
- **refresh_tokens:** id, user_id, token, expiry_date, is_valid, created_at

7.0 UI/UX DESIGN

7.1 Web Application Wireframe



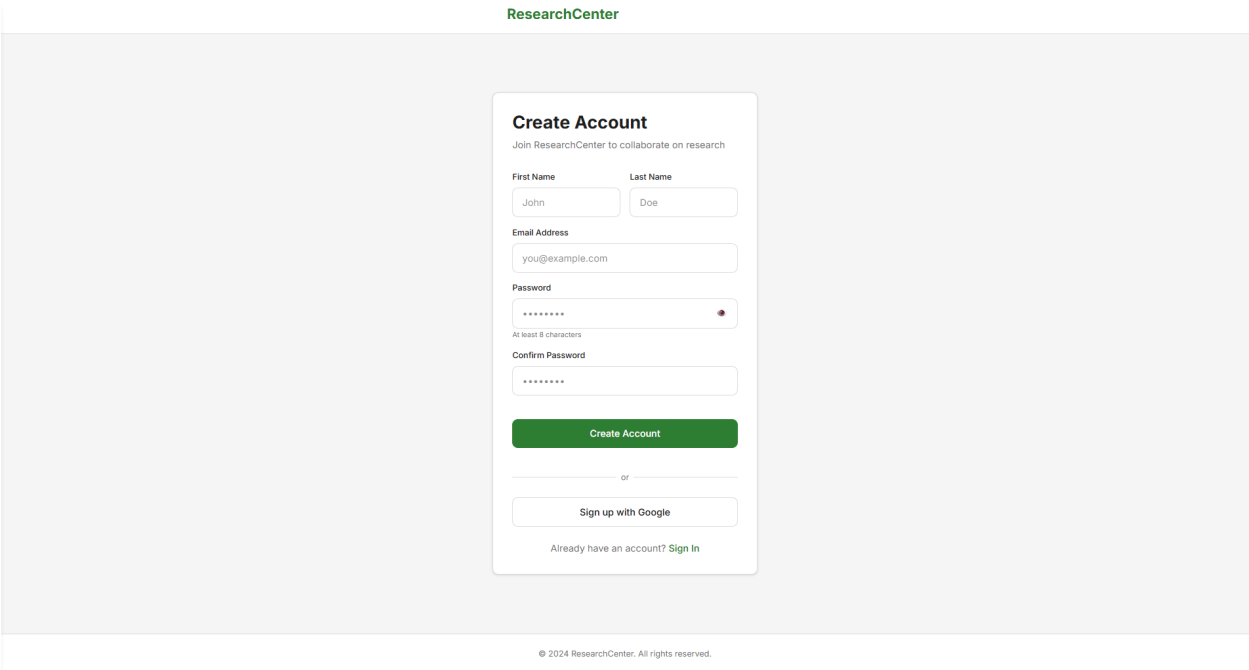
Login Page

Header: [Logo] ResearchCenter
Content:

- Email field
- Password field
- "Sign In" button
- "Sign Up" link
- "Login with Google" button

- "Forgot Password?" link

Footer: Links, Copyright

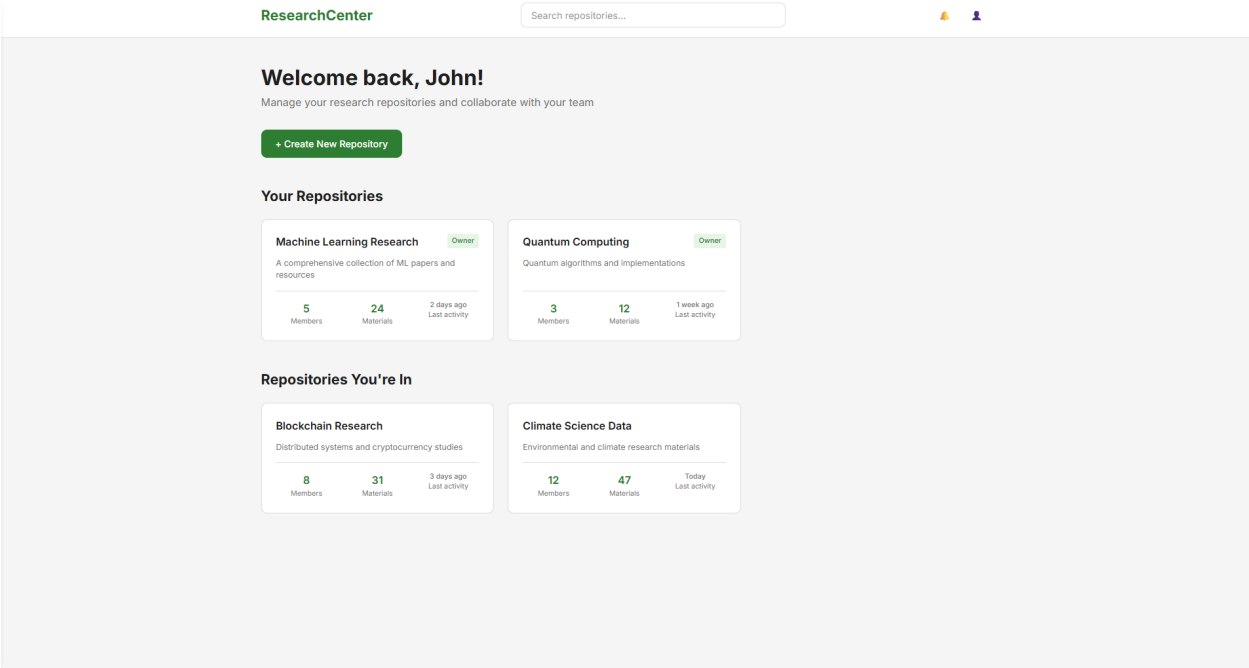


Registration Page

Header: [Logo] ResearchCenter
Content:

- Email field
- Password field
- Confirm Password field
- First Name field
- Last Name field
- "Create Account" button
- "Login with Google" button
- "Already have an account? Sign In" link

Footer: Links, Copyright



Repositories Dashboard

Header: [Logo] ResearchCenter [Search] [Notifications] [Profile Dropdown]

Content:

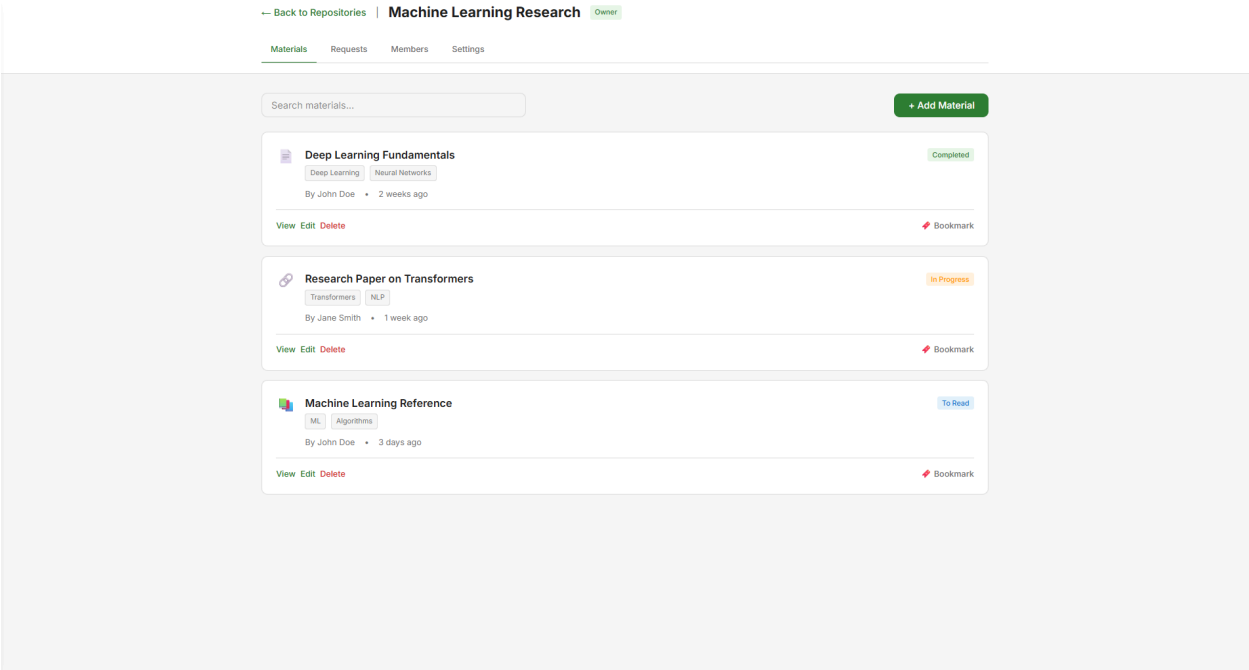
- Welcome message: "Welcome back, John!"
- "Create New Repository" button

Your Repositories section:

- Repository cards in grid (3 columns)
- Each card shows:
    - Repository name
    - Description (truncated)
    - Owner badge (if owner)
    - Member count
    - Material count
    - Last activity
    - "Open" button

Repositories You're In section:

- Similar cards for repositories where user is member



Repository Details

Header: [← Back to Repositories] [Repository Name] [Owner/Member badge]

Tabs: [Materials] [Requests] [Members] [Settings] (owner only)

Materials Tab:

- Search bar
- Filter by tags/status
- "Add Material" button
- Material grid with:
    - Icon (PDF/Link/Reference)
    - Title
    - Tags
    - Status badge
    - Uploader

    - Date
    - Actions: View, Edit (if owner/uploader), Delete (if owner/uploader), Bookmark

Requests Tab:

- "New Request" button
- Tabs: Open | Fulfilled | My Requests
- Request cards with:
    - Title
    - Requester
    - Date
    - Status badge
    - Description (truncated)
    - "Fulfill" button (if open)
    - "View Material" button (if fulfilled)

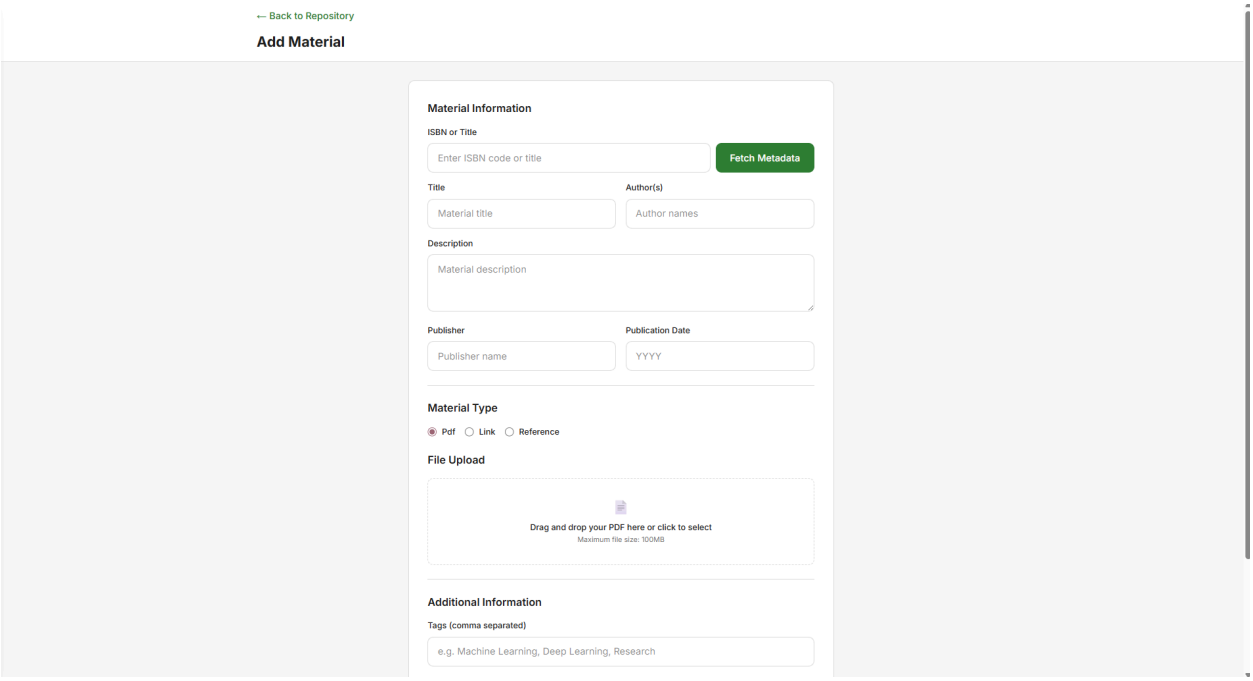
Members Tab:

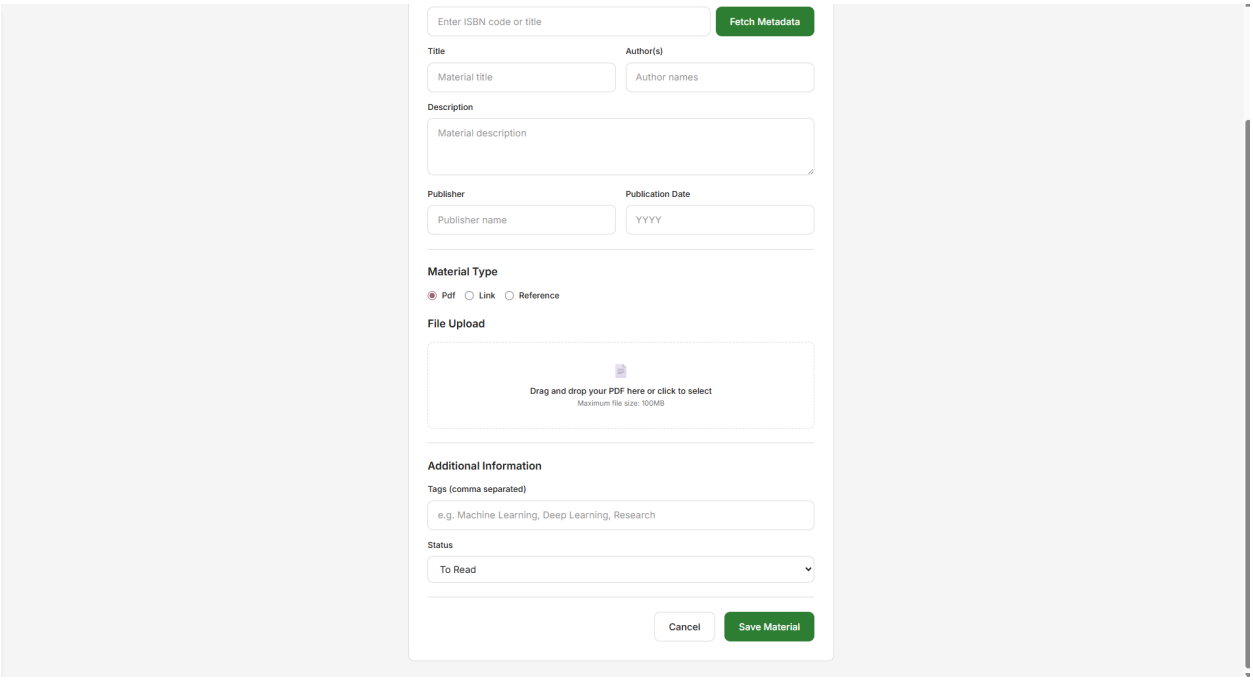
- List of members with:
    - Name
    - Email
    - Role (Owner/Member)
    - Joined date
    - "Remove" button (owner only)
    - "Change Role" dropdown (owner only)
- "Invite Member" button (owner only) with email input

Settings Tab (owner only):

- Repository name edit
- Description edit

- Visibility toggle (Public/Private)
- "Delete Repository" button (with confirmation)



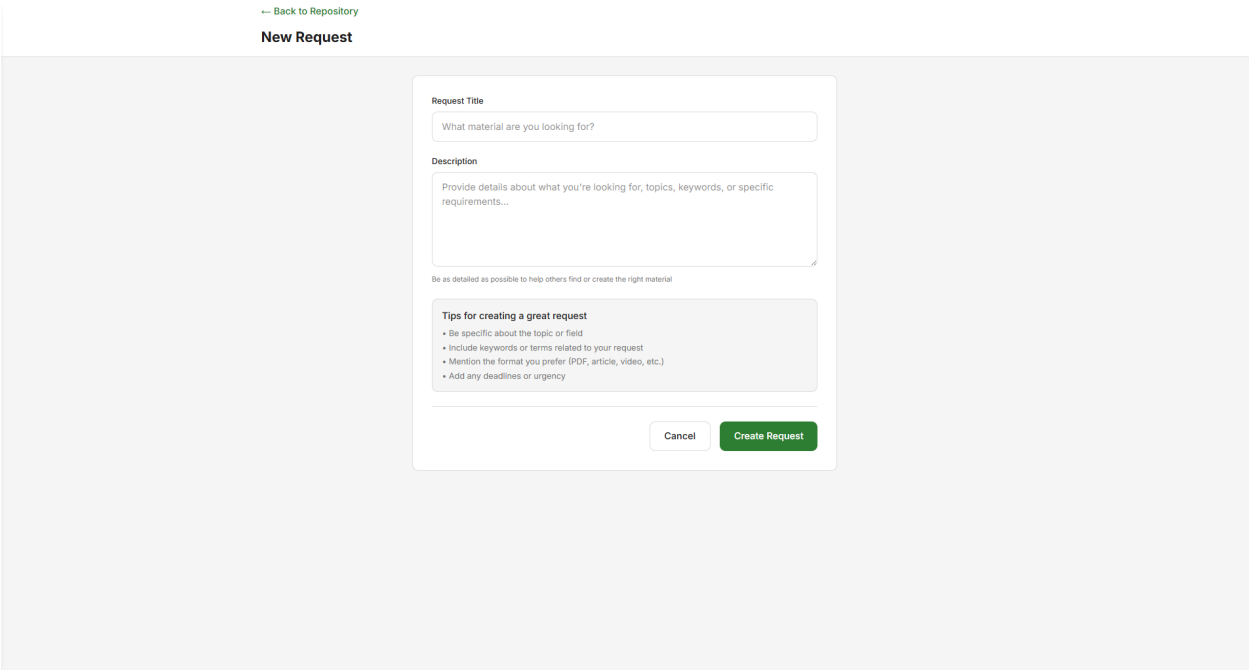


Add Material Page

Header: [← Back to Repository] Add Material

Content:

- ISBN/Title search with "Fetch Metadata" button
- Auto-populated fields:
    - Title
    - Author(s)
    - Description
    - Publisher
    - Publication Date
- Material Type selector (PDF/Link/Reference)
- File upload area (if PDF selected) with drag-and-drop
- URL field (if Link selected)
- Tags input (comma separated)
- Status dropdown (To Read/In Progress/Completed)
- "Save Material" button
- "Cancel" button

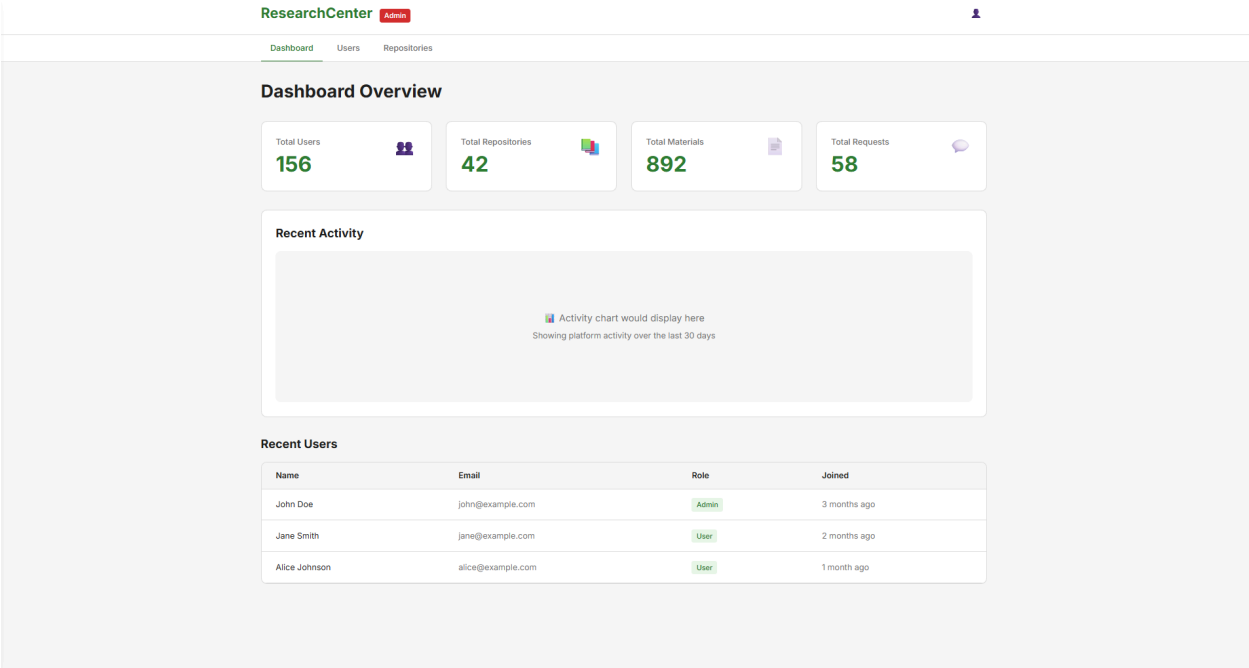


New Request Page

Header: [← Back to Repository] New Request

Content:

- Title field
- Description field (textarea)
- "Create Request" button
- "Cancel" button



Admin Dashboard

Header: [Logo] ResearchCenter [Admin] [Profile Dropdown]

Sidebar: [Dashboard] [Users] [Repositories] [Materials] [Requests]

Dashboard Tab:

- Statistics cards:
    - Total Users
    - Total Repositories
    - Total Materials
    - Total Requests

- Recent activity graph
- Recent users list

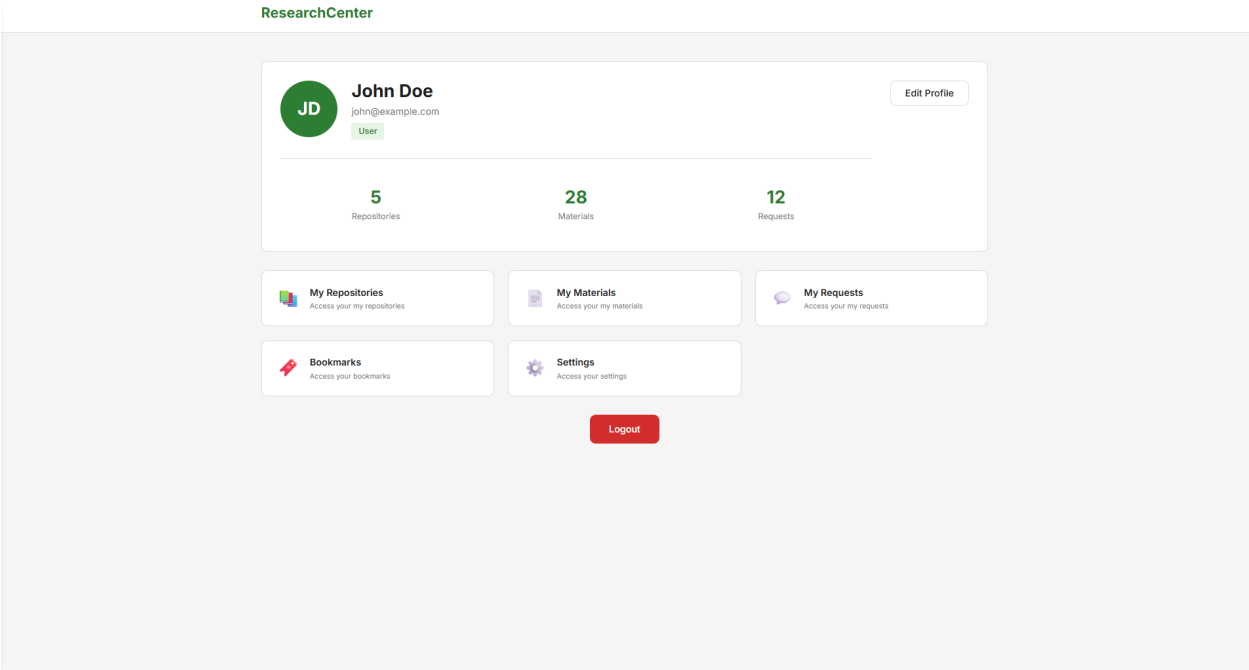
Users Tab:

- Search bar
- User table with:
    - ID
    - Name
    - Email
    - Role
    - Created date
    - Actions: Edit Role, Suspend, Delete

Repositories Tab:

- List of all repositories with:
    - Name
    - Owner
    - Visibility
    - Member count
    - Material count

Created dateActions: View, Delete



Profile Page

Header: [←] Back | [Logo] ResearchCenter | [⚙️] Settings

Content:

- Profile picture (avatar with initials)
- Name field
- Email field
- Institution field
- Member since date
- Bio field
- Research interests tags
- Website URL field
- Location field

Account Settings:

- Change password button
- Email notifications toggle

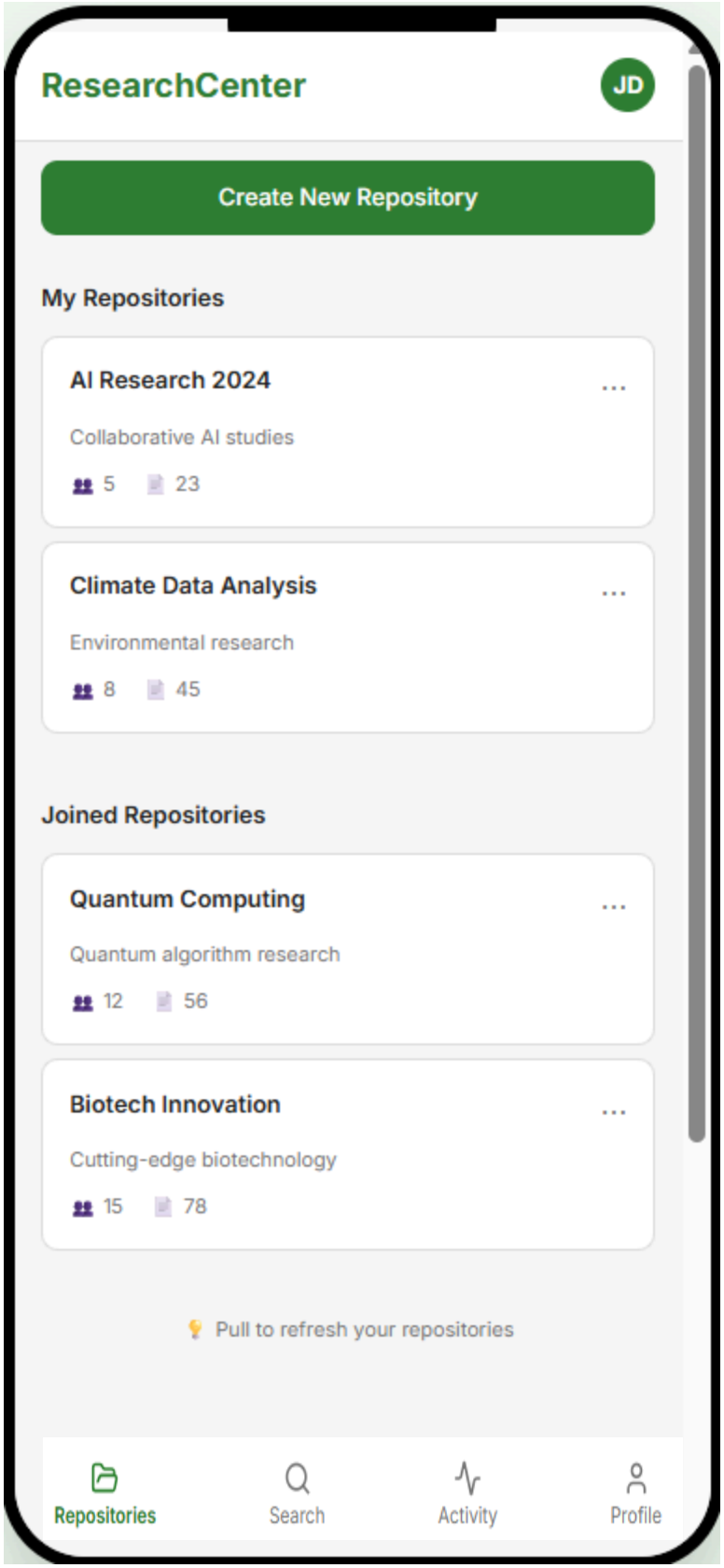
- Two-factor authentication toggle

Footer: Links, Copyright

7.2 Mobile Application Wireframes

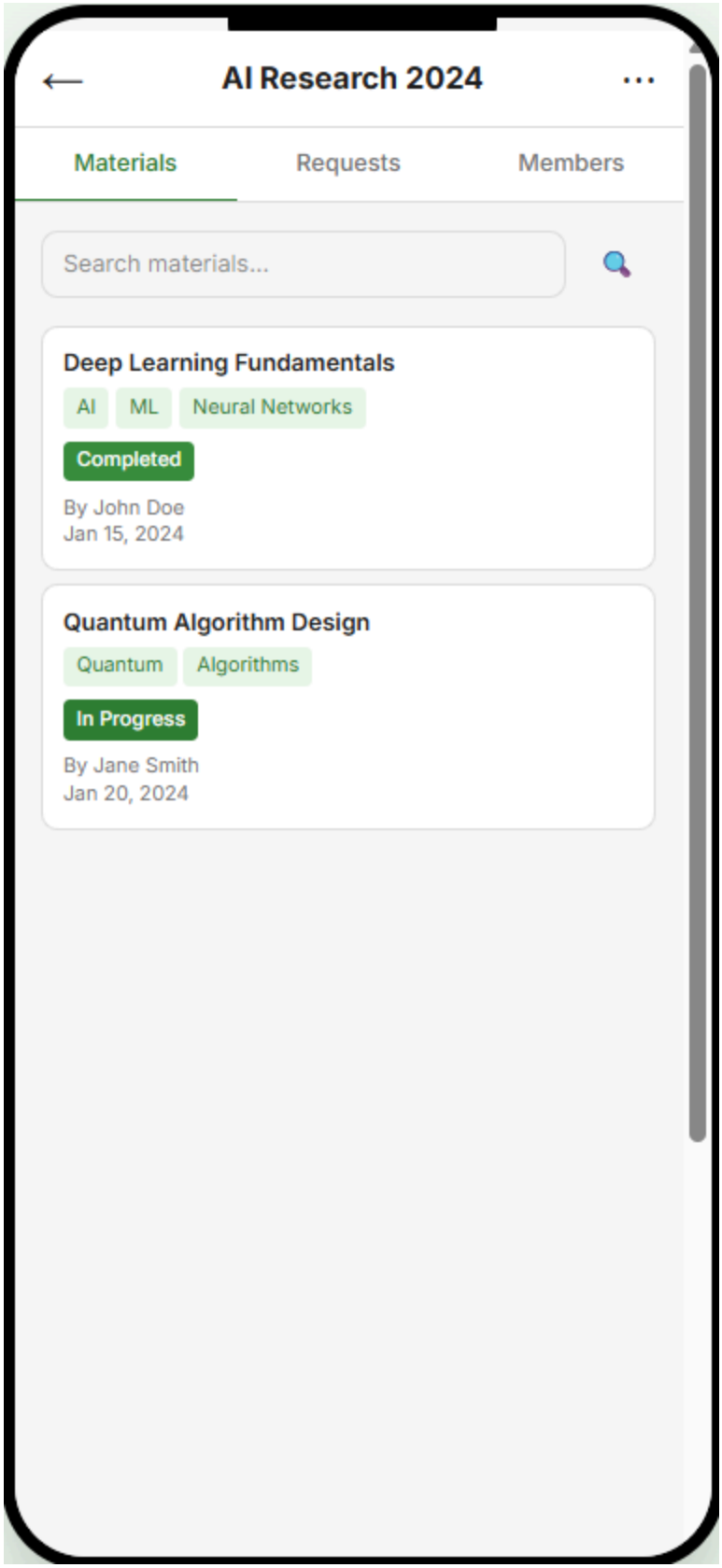
Bottom Navigation

- [Repositories] [Search] [Activity] [Profile]


ResearchCenter
JD
Create New Repository
My Repositories
AI Research 2024
Collaborative AI studies
5
23
Climate Data Analysis
Environmental research
8
45
Joined Repositories
Quantum Computing
Quantum algorithm research
12
56
Biotech Innovation
Cutting-edge biotechnology
15
78
Pull to refresh your repositories
Repositories
Search
Activity
Profile

Repositories Screen

- Header: "ResearchCenter" with profile icon
- "Create New Repository" button (Primary Green #2E7D32)
- Section: "My Repositories"
    - Vertical list of owned repositories
    - Each item: Name, Description, Member count, Material count
- Section: "Joined Repositories"
    - Vertical list of repositories user is member of
    - Each item: Name, Description, Member count, Material count
- Pull to refresh
- Swipe left to archive/delete (owner only)


AI Research 2024
Materials
Requests
Members
Search materials...
Deep Learning Fundamentals
AI
ML
Neural Networks
Completed
By John Doe
Jan 15, 2024
Quantum Algorithm Design
Quantum
Algorithms
In Progress
By Jane Smith
Jan 20, 2024

Repository Detail Screen

- Header: [←] Repository Name with menu [···]
- Tabs: [Materials] [Requests] [Members]

Materials Tab

- Floating Action Button: [+] "Add Material"
- Search bar
- Material list (vertical):
    - Title
    - Tags
    - Status badge (To Read/In Progress/Completed)
    - Uploader name
    - Date added
- Swipe left for quick actions (Edit, Delete, Bookmark)

Requests Tab

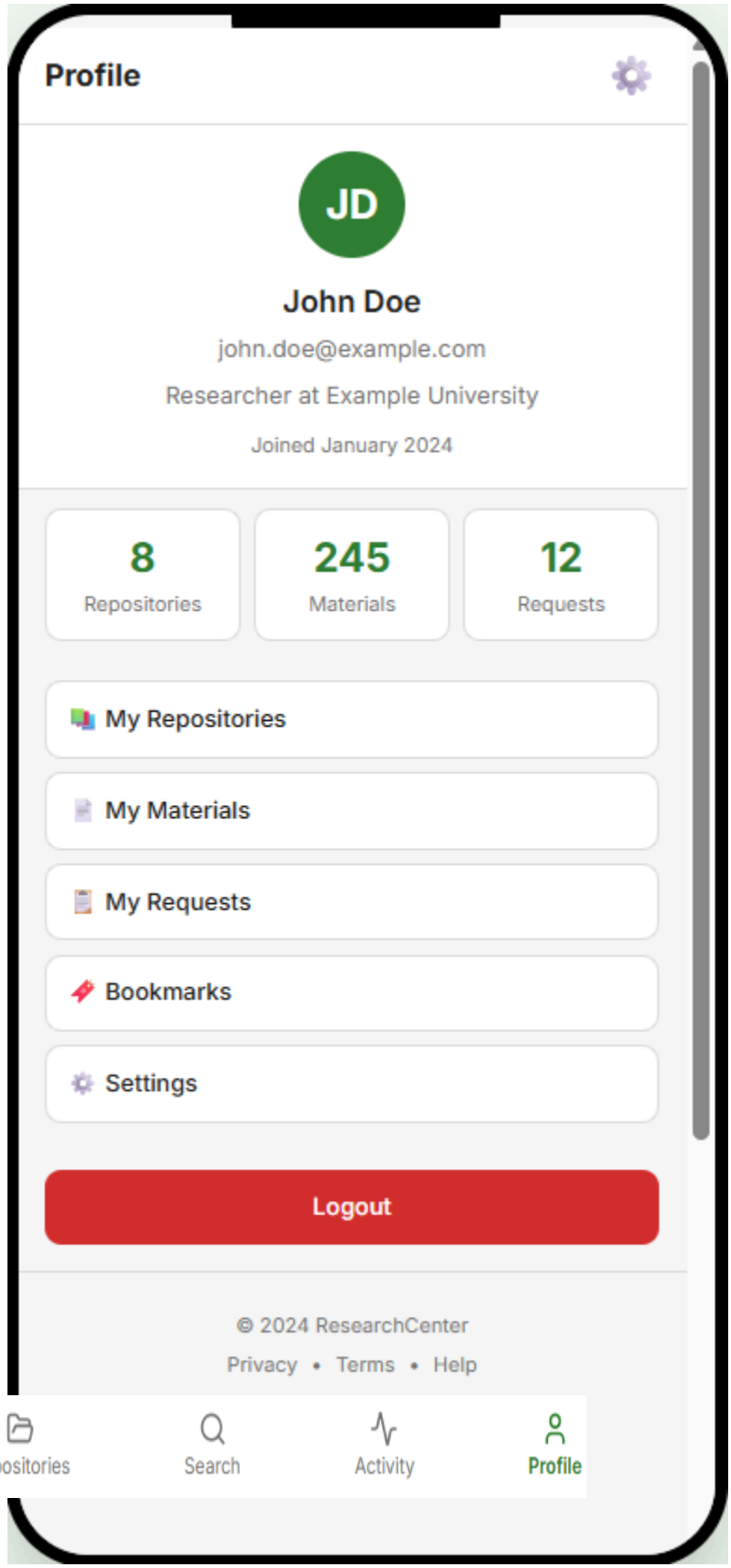
- Floating Action Button: [+] "New Request"
- Segmented controls: [Open] [Fulfilled]
- Request cards:
    - Title
    - Requester name
    - Date
    - Status badge
    - Description (truncated)
    - "Fulfill" button (if open)
    - "View Material" button (if fulfilled)

Members Tab

- Member list with avatars

- Each item: Name, Email, Role (Owner/Member), Joined date

- "Invite Member" button (owner only)



Profile Screen

- Header: "Profile"
- User avatar
- Name
- Email
- Role
- Statistics cards: Repositories, Materials, Requests
- Menu items:
    - "My Repositories"
    - "My Materials"
    - "My Requests"
    - "Bookmarks"
    - "Settings"
    - "Logout" (Error Red #D32F2F)

Mobile-Specific Features:

- Touch-optimized buttons (min 44×44px)
- Gesture support (swipe, pull-to-refresh)
- Bottom navigation for main actions
- Floating Action Button for primary actions
- Simplified forms for mobile input
- Haptic feedback on key actions

Design System:

- Colors: Primary Green (#2E7D32), Secondary Green (#4CAF50), Success (#388E3C), Error (#D32F2F), Warning (#FFA000), Background (#F5F5F5), Surface (#FFFFFF), Text Primary (#212121), Text Secondary (#757575)
- Typography: Inter font family, responsive sizing

- Spacing: 8px grid system
- Components: Consistent buttons, inputs, cards, modals
- Responsive: Mobile-first approach, breakpoints at 640px, 768px, 1024px
- WCAG 2.1 AA compliant color contrast (minimum 4.5:1)

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design

Day 5-6: UI/UX wireframes

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-5)

Week 3: Foundation

Day 1: Spring Boot setup with dependencies (Security, JPA, Web, WebSocket)

Day 2: Database configuration and entity classes

Day 3: JWT authentication implementation

Day 4: User registration and login endpoints

Day 5: Google OAuth integration

Week 4: Core Features - Part 1

Day 1: Repository CRUD endpoints

Day 2: Repository membership and invitations

Day 3: Material CRUD endpoints

Day 4: File upload to Supabase Storage

Day 5: Google Books API integration

Week 5: Core Features - Part 2

Day 1: Material Request endpoints

Day 2: Email service (SMTP) implementation

Day 3: WebSocket configuration and real-time events

Day 4: Activity logging system

Day 5: Admin endpoints and error handling

Phase 3: Web Application (Week 6-7)

Week 6: Frontend Foundation

Day 1: React + Vite setup with TypeScript, Tailwind CSS

Day 2: Authentication pages (login, register, Google OAuth)

Day 3: Repositories dashboard

Day 4: Repository detail view with tabs

Day 5: Add Material page with Google Books fetch

Week 7: Complete Web Features

Day 1: Requests board with real-time updates

Day 2: Request creation and fulfillment UI

Day 3: Members management and invitations

Day 4: User profile and bookmarks

Day 5: Admin dashboard

Phase 4: Mobile Application (Week 8-9)

Week 8: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Authentication screens

Day 3: Product browsing

Day 4: Shopping cart

Day 5: API service layer

Week 8: Complete Mobile App

Day 1: Add Material screen with file upload

Day 2: Requests board with tabs

Day 3: Request creation and fulfillment

Day 4: Profile screen and settings

Day 5: Testing and APK generation

Phase 5: Integration & Deployment (Week 10)

Week 10: Deployment

Day 1: End-to-end testing across platforms

Day 2: Bug fixes and optimization

Day 3: Security review

Day 4: Backend deployment (Render), Web deployment (Vercel)

Day 5: Final testing and project submission

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 5):** Backend API fully functional
- **M3 (End Week 7):** Web application complete
- **M4 (End Week 9):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Repository management (Week 4)
3. Material and Request systems (Week 5)
4. Web dashboard (Week 6-7)
5. Mobile app (Week 8-9)
6. Cross-platform testing (Week 10)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements
- Implement caching for external API calls to handle rate limits
- Set file size limits (10MB) and validate file types
- Provide fallback to polling if WebSocket connections fail
- Use queue system with retry mechanism for email delivery
- Add database indexes on frequently queried columns
- Use unique constraints and transaction isolation for data integrity